

Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

ОТЧЁТ

**по индивидуальному заданию к лабораторной работе №6
по дисциплине “Системное программное обеспечение GNU/Linux”
Ускорение сборки ядра с помощью tmpfs**

Выполнил
студент гр. 5130904/50005

Стойко Е. А.

Руководитель

Петров А. В.

Санкт-Петербург

2026 г.

Содержание

1. Цель	3
2. Задачи	3
3. Сведения о системе	3
3.1. Аппаратная платформа	3
3.2. Программная платформа	3
4. Выполнение работы	4
4.1. Подготовка системы	4
4.2. Получение исходного кода ядра	4
4.3. Конфигурация ядра	4
4.4. Сборка без tmpfs	4
4.5. Подготовка tmpfs	4
4.6. Сборка с tmpfs	5
5. Анализ результатов сборки	5
6. Вывод	6
Список использованных источников	6

Операционные системы семейства Linux позволяют гибко настраивать и оптимизировать процесс сборки ядра. Одним из способов ускорения компиляции является использование файловой системы tmpfs — временной файловой системы, хранящей данные непосредственно в оперативной памяти. Поскольку скорость доступа к ОЗУ на порядок превышает скорость дисковых операций, размещение дерева исходных файлов в tmpfs способно существенно сократить время сборки.

1. Цель

Сборка ядра Linux без символьной информации с размещением дерева исходных файлов в файловой системе tmpfs, а также данные об ускорении времени сборки по сравнению со стандартной сборкой на дисковой файловой системе.

2. Задачи

1. Подготовить систему к сборке ядра.
2. Установить исходные коды ядра дистрибутива.
3. Выполнить конфигурацию ядра (без символьной информации).
4. Собрать ядро без использования tmpfs и замерить время сборки.
5. Смонтировать tmpfs и разместить дерево исходных файлов в ней.
6. Собрать ядро с использованием tmpfs и замерить время сборки.
7. Вычислить ускорение и проанализировать результаты.

3. Сведения о системе

3.1. Аппаратная платформа

Архитектура: x86_64

Процессор: AMD Ryzen 7 7700 8-Core Processor @ 3.80GHz

Ядра: 8

Потоки: 16

ОЗУ: 32 GiB

3.2. Программная платформа

ОС: Ubuntu 24.04.4 LTS (Noble Numbat)

Virtual Machine ОЗУ: 24 GiB

4. Выполнение работы

4.1. Подготовка системы

Были установлены необходимые для сборки зависимости:

```
sudo apt update && \
    sudo apt build-dep -y linux linux-image-unsigned-$(uname -r) && \
    sudo apt install -y fakeroot llvm libncurses-dev dwarves
```

4.2. Получение исходного кода ядра

Для получения возможности загрузить исходный код ядра строка `deb-src` была добавлена в файл `/etc/apt/sources.list.d/ubuntu.sources`.

Исходный код ядра был получен из репозитория дистрибутива:

```
sudo apt source linux-image-unsigned-$(uname -r)
```

4.3. Конфигурация ядра

Во избежание конфликтов ядер в `Makefile` было изменено имя версии ядра на `custom`:

```
VERSION = 6
PATCHLEVEL = 14
SUBLEVEL = 11
EXTRAVERSION = -custom
```

Для конфигурации использовалась текущая конфигурация системы:

```
cp /boot/config-$(uname -r) .config
make menuconfig
```

Для ускорения сборки была отключена отладочная (символьная) информация:

```
scripts/config --disable DEBUG_INFO
scripts/config --disable DEBUG_INFO_DWARF_TOOLCHAIN_DEFAULT
```

4.4. Сборка без `tmpfs`

Перед замером времени был очищен дисковый кэш операционной системы (*page cache*), чтобы исключить влияние кэшированных данных на результат измерения:

```
sudo sync
echo 3 | sudo tee /proc/sys/vm/drop_caches
```

Сборка выполнялась с использованием цели сборки `bindeb-pkg`:

```
time make -j8 bindeb-pkg
```

4.5. Подготовка `tmpfs`

Была создана точка монтирования и смонтирована файловая система `tmpfs` объёмом 14 ГБ [1]:

```
sudo mount -t tmpfs -o size=14G tmpfs /home/host/lab6/
```

Проверяем:

```
df -hT /home/host/lab6/
```

Filesystem	Type	Size	Used	Avail	Use%	Mounted on
tmpfs	tmpfs	14G	0	14G	0%	/home/host/lab6

Аналогично первому этапу, перед сборкой был очищен дисковый кэш:

```
sudo sync; echo 3 | sudo tee /proc/sys/vm/drop_caches
```

4.6. Сборка с tmpfs

```
time make -j8 bindeb-pkg
```

По завершении работы tmpfs была размонтирована для освобождения оперативной памяти [1]:

```
sudo umount /home/host/lab6/
```

5. Анализ результатов сборки

Тип	Время
Без tmpfs	27m 8.139s
С tmpfs	20m 20.898s

По формуле $\frac{T-W}{T} \cdot 100\%$, где T - время с tmpfs, а W - без, посчитаем насколько первый вариант быстрее.

$$\begin{aligned}\frac{T-W}{T} \cdot 100\% &= \frac{(27 \cdot 60 + 8.139) - (20 \cdot 60 + 20.898)}{27 \cdot 60 + 8.139} \cdot 100\% = \\ &= \frac{1628.139 - 1220.898}{1628.139} \cdot 100\% \approx 25.01\%\end{aligned}$$

Таким образом, использование tmpfs позволило сократить время сборки на 25%.

Наблюдаемый эффект объясняется тем, что при хранении исходных файлов на диске (HDD) значительная часть времени тратится на операции ввода-вывода. Компилятор GCC выполняет большое количество мелких операций чтения заголовочных файлов и записи объектных файлов. Дисковый кэш Linux частично нивелирует эту задержку, однако после принудительной очистки кэша преимущество tmpfs проявляется в полной мере.

6. Вывод

В ходе лабораторной работы была достигнута поставленная цель: ядро Linux собрано как на дисковой файловой системе, так и с размещением дерева исходных файлов в файловой системе tmpfs, и определено ускорение сборки.

Были решены следующие задачи:

- система подготовлена к сборке
- получен и сконфигурирован исходный код ядра без символьной информации
- выполнены замеры времени в двух условиях
- вычислено ускорение

Получены следующие показатели: время сборки на диске составило 27 минут, время сборки в tmpfs — 20 минут, то есть сборка в tmpfs выполнялась на 25% быстрее.

В результате работы получено практическое понимание принципов работы tmpfs и её влияния на производительность компиляции. Выяснено, что эффективность tmpfs определяется соотношением скорости дискового ввода-вывода и объёма доступной оперативной памяти, а также корректностью методологии замеров времени.

Список использованных источников

1. Manpages RU. Ubuntu Manpage: tmpfs [Электронный ресурс]. 2026. URL: <https://manpages.ubuntu.com/manpages/noble/ru/man5/tmpfs.5.html>.